

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Динамические списки»

Выполнил: ст. гр. 25BBB1
Какулин В.С.
Самсонов М.А.

Проверил: к.т.н. доцент
Юрова О.В.
к.т.н., ст. преподаватель
Деев М.В.

Пенза 2026

Название:

Динамические списки.

Цель работы:

Изучение динамических структур данных и реализация приоритетной очереди, классической очереди и стека.

Лабораторное задание:

- 1) Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
- 2) На основе приведенного кода реализуйте структуру данных Очередь.
- 3) На основе приведенного кода реализуйте структуру данных Стек.

Ход работы:

Задание 1:

Описание используемого алгоритма:

Этот алгоритм реализует приоритетную очередь, где каждый элемент хранится вместе со своим приоритетом.

Краткое описание работы ключевых функций:

push (добавление с учетом приоритета): Создает новый узел и ищет для него правильное место. Если у нового элемента приоритет выше, чем у головы, он встает в самое начало. Иначе алгоритм продвигается вперед, пока не найдет узел с меньшим или равным приоритетом, и вклинивает новый элемент между ними, сохраняя сортировку.

pop (удаление): Извлекает элемент из головы (где всегда находится элемент с наивысшим приоритетом), выводит его данные и освобождает выделенную память.

printQueue (просмотр): Последовательно обходит все элементы от начала до конца, выводя их значения и приоритеты.

main: Организует цикл интерактивного ввода данных до команды **stop**, после чего предоставляет меню для добавления новых элементов, извлечения приоритетного элемента или вывода текущего состояния.

Результат выполнения программы:

```
● vlad@MacBook-Air lab3 % cd "/Users/vlad/Documents/lab3/" && gcc task1.c -o
  ents/lab3/"task1
  === Ввод элементов очереди ===
  Введите значение элемента (или 'stop' для завершения ввода):
  Значение: 100
  Приоритет: 1
  Элемент добавлен. Продолжайте ввод или введите 'stop'.

  Значение: 12
  Приоритет: 4
  Элемент добавлен. Продолжайте ввод или введите 'stop'.

  Значение: 23
  Приоритет: 10
  Элемент добавлен. Продолжайте ввод или введите 'stop'.

  Значение: 56 0
  Приоритет: Элемент добавлен. Продолжайте ввод или введите 'stop'.

  Значение: stop
  Содержимое упорядоченной очереди:
  Значение: 23 | Приоритет: 10
  Значение: 12 | Приоритет: 4
  Значение: 100 | Приоритет: 1
  Значение: 56 | Приоритет: 0

  Выберите действие:
  1. Добавить элемент (push)
  2. Удалить элемент с наивысшим приоритетом (pop)
  3. Показать очередь (print)
  0. Выход
  Ваш выбор: 2
  Удален элемент со значением: 23 и приоритетом: 10

  Выберите действие:
  1. Добавить элемент (push)
  2. Удалить элемент с наивысшим приоритетом (pop)
  3. Показать очередь (print)
  0. Выход
  Ваш выбор: 3
  Содержимое упорядоченной очереди:
  Значение: 12 | Приоритет: 4
  Значение: 100 | Приоритет: 1
  Значение: 56 | Приоритет: 0

  Выберите действие:
  1. Добавить элемент (push)
  2. Удалить элемент с наивысшим приоритетом (pop)
  3. Показать очередь (print)
  0. Выход
  Ваш выбор: 0
```

Рисунок 1 – Результат выполнения программы задания 1.

Задание 2:

Описание используемого алгоритма:

Этот алгоритм реализует классическую структуру данных Очередь, работающую по принципу FIFO, с использованием управляющей структуры Queue, которая содержит указатели на начало head и конец tail для эффективного добавления элементов.

Краткое описание работы ключевых функций:

push (добавление): Отвечает за создание нового узла со значением и помещение его в хвост очереди, автоматически обновляя указатель tail, а если очередь изначально была пустой, то и head.

Pop (удаление): Выполняет извлечение элемента из начала очереди, сдвигает head на следующий узел, освобождает выделенную память, а при опустошении очереди обнуляет tail.

printQueue (просмотр): Последовательно обходит элементы от начала до конца через вспомогательный указатель current и выводит их значения в консоль.

main: Организует интерактивный цикл заполнения элементов до команды stop, после чего предоставляет пользовательское меню с возможностью добавления новых элементов, их удаления, просмотра состояния очереди и корректного завершения программы с полной очисткой памяти.

Результат выполнения программы:

```

vlad@MacBook-Air lab3 % cd "/Users/vlad/Documents/lab3/" && gcc task2.c -o t
=== Ввод элементов очереди ===
Введите значение элемента (или 'stop' для завершения ввода):
Значение: 1
Элемент добавлен. Продолжайте ввод или введите 'stop'.

Значение: 2
Элемент добавлен. Продолжайте ввод или введите 'stop'.

Значение: 3
Элемент добавлен. Продолжайте ввод или введите 'stop'.

Значение: 4
Элемент добавлен. Продолжайте ввод или введите 'stop'.

Значение: stop
Содержимое очереди:
Значение: 1
Значение: 2
Значение: 3
Значение: 4

Выберите действие:
1. Добавить элемент (push)
2. Удалить элемент (pop)
3. Показать очередь (print)
0. Выход
Ваш выбор: 2
Удален элемент со значением: 1

Выберите действие:
1. Добавить элемент (push)
2. Удалить элемент (pop)
3. Показать очередь (print)
0. Выход
Ваш выбор: 3
Содержимое очереди:
Значение: 2
Значение: 3
Значение: 4

Выберите действие:
1. Добавить элемент (push)
2. Удалить элемент (pop)
3. Показать очередь (print)
0. Выход
Ваш выбор: 0

```

Рисунок 2 – Результат выполнения программы задания 2.

Задание 3:

Описание используемого алгоритма:

Этот алгоритм реализует структуру данных Стек, работающую по принципу LIFO, с использованием единственного указателя `head`, который всегда указывает на вершину структуры.

Краткое описание работы ключевых функций:

push (добавление): Создает новый узел со значением, связывает его с текущей вершиной через поле `next` и переносит указатель `head` на этот новый элемент, помещая его на самый верх.

pop (удаление): Извлекает верхний элемент, перенаправляет `head` на следующий за ним узел и освобождает выделенную под удаленный элемент память.

printStack (просмотр): Осуществляет последовательный обход элементов от вершины к дну с помощью вспомогательного указателя current и выводит их значения на экран.

main: Настраивает интерактивный цикл ввода данных до тех пор, пока пользователь не введет слово stop, после чего открывает меню управления для добавления, удаления, просмотра стека и завершения программы с очисткой памяти.

Результат выполнения программы:

```
vlad@MacBook-Air lab3 % cd "/Users/vlad/Documents/lab3/" && gcc task3.c -o task3
=== Ввод элементов стека ===
Введите значение элемента (или 'stop' для завершения ввода):
Значение: 1
Элемент добавлен на вершину стека. Продолжайте ввод или введите 'stop'.

Значение: 2
Элемент добавлен на вершину стека. Продолжайте ввод или введите 'stop'.

Значение: 3
Элемент добавлен на вершину стека. Продолжайте ввод или введите 'stop'.

Значение: 4
Элемент добавлен на вершину стека. Продолжайте ввод или введите 'stop'.

Значение: stop
Содержимое стека (от верхушки к дну):
Значение: 4
Значение: 3
Значение: 2
Значение: 1

Выберите действие:
1. Добавить элемент в стек (push)
2. Удалить элемент из стека (pop)
3. Показать стек (print)
0. Выход
Ваш выбор: 2
Удален верхний элемент со значением: 4

Выберите действие:
1. Добавить элемент в стек (push)
2. Удалить элемент из стека (pop)
3. Показать стек (print)
0. Выход
Ваш выбор: 3
Содержимое стека (от верхушки к дну):
Значение: 3
Значение: 2
Значение: 1

Выберите действие:
1. Добавить элемент в стек (push)
2. Удалить элемент из стека (pop)
3. Показать стек (print)
0. Выход
Ваш выбор: 0
```

Рисунок 3 – Результат выполнения программы задания 3.

Вывод:

В ходе выполнения лабораторной работы были изучены динамические структуры данных и реализованы программы для приоритетной очереди, классической очереди и стека. Написанный код полностью работоспособен, корректно выполняет все необходимые операции (добавление, удаление и просмотр элементов) и правильно управляет динамической памятью.